Rails Matchers Cheat Sheet

In addition to these spec types, rspec-rails provides some Rails-specific matchers. Some of these are available to any spec once you've required rails_helper; others are just for certain spec types.

Matcher	Passes if…	Available in…
be_a_new(model_class)	record.is_a(model_class) && record.new_record?	All spec types
be_a_new(model_class).with(attribute: 'value')	record.is_a(model_class) && record.new_record? && record.attribute == 'value'	All spec types
be_new_record	record.new_record?	All spec types
be_valid	record.valid?	All spec types
have_been_enqueued.with(some_args)	Job was enqueued with matching arguments	All spec types
have_enqueued_job(job_class).with(some_args), enqueue_job(job_class).with(some_args)	Code in an expect block enqueued a job_class job with matching arguments	All spec types
have_http_status(code)	response.status == code	All spec types
have_http_status(symbol)	response.status maps to a *kind* of response, such as :success	All spec types
redirect_to('http://some-url.example.com')	Response redirects to the specified URL	:request, :controller
be_routable	Route exists	:controller, :routing
route_to(controller: 'name', action: 'name'), route_to('controller#action')	Route leads to the specified controller/action/parameters	:controller, :routing

Please don't feel like you need to use all of these spec types in the same app! Here are a few recommendations for specific situations.

When you're doing outside-in acceptance testing:

- For HTTP-based APIs, use request specs.
- For user-facing web applications, add Capybara to the project and use feature specs; see Michael Crismali's article for setup advice.[7]

7. https://www.devmynd.com/blog/setting-up-rspec-and-capybara-in-rails-5-for-testing/